# DATA STRUCTURE – C PROGRAMS

Roll No.: 2025115042

## Questions, Programs and Sample Outputs

The following 10 programs are prepared from the handwritten questions provided. Programs are written in C and include a sample output for each question.

## 1. Integer array and pointer arithmetic

**Question:** Define an integer array. Declare an integer pointer and initialize it to point to the first element of the array. Use pointer arithmetic to traverse the array and print all elements.

### *Program*

```
#include <stdio.h>

int main() {
    int a[] = {10, 20, 30, 40, 50};
    int n = sizeof(a) / sizeof(a[0]);
    int *p = a;

    printf("Array elements: ");
    for (int i = 0; i < n; i++)
        printf("%d ", *(p + i));

    printf("\\n");
    return 0;
}
```

### *Output*

```
Array elements: 10 20 30 40 50
```

## 2. Modify array elements using pointer arithmetic

**Question:** Modify the elements of an array using pointer arithmetic.

### *Program*

```
#include <stdio.h>

int main() {
    int a[] = {10, 20, 30, 40, 50};
    int n = sizeof(a) / sizeof(a[0]);
    int *p = a;

    for (int i = 0; i < n; i++)
        *(p + i) = *(p + i) * 2;

    printf("Modified array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", *(p + i));

    printf("\\n");
    return 0;
}
```

### *Output*

```
Modified array: 20 40 60 80 100
```

## 3. Calculate string length using a pointer

**Question:** Calculate the length of a string using a pointer.

### *Program*

```
#include <stdio.h>

int main() {
    char str[] = "Data Structure";
    char *p = str;
    int length = 0;

    while (*p != '\0') {
        length++;
        p++;
    }

    printf("String: %s\\n", str);
    printf("Length = %d\\n", length);

    return 0;
}
```

### *Output*

```
String: Data Structure
Length = 14
```

## 4. Function to calculate sum and product using pointers

**Question:** Write a C function that calculates both the sum and product of two numbers. Return both results using pointers to the function arguments.

### *Program*

```
#include <stdio.h>

void calculate(int a, int b, int *sum, int *product) {
    *sum = a + b;
    *product = a * b;
}

int main() {
    int a = 8, b = 5;
    int sum, product;

    calculate(a, b, &sum, &product);

    printf("First number  = %d\\n", a);
    printf("Second number = %d\\n", b);
    printf("Sum           = %d\\n", sum);
    printf("Product       = %d\\n", product);

    return 0;
}
```

### *Output*

```
First number  = 8
Second number = 5
Sum           = 13
Product       = 40
```

## 5. Array of structures with dynamic memory and structure pointer

**Question:** Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer to iterate and access/modify the members.

### *Program*

```
#include <stdio.h>
#include <stdlib.h>

struct Student {
    int id;
    char name[20];
    float mark;
};

int main() {
    int n = 3;
    struct Student *p;

    p = (struct Student *)malloc(n * sizeof(struct Student));

    if (p == NULL) {
        printf("Memory allocation failed.\\n");
        return 1;
    }

    p[0] = (struct Student){101, "Arun", 85.5};
    p[1] = (struct Student){102, "Bala", 78.0};
    p[2] = (struct Student){103, "Kavin", 91.0};

    /* Modify a member using the structure pointer */
    p[1].mark = 82.0;

    printf("Student details:\\n");
    for (int i = 0; i < n; i++) {
        printf("%d  %s  %.1f\\n",
               (p + i)->id, (p + i)->name, (p + i)->mark);
    }

    free(p);
    return 0;
}
```

### *Output*

```
Student details:
101  Arun  85.5
102  Bala  82.0
103  Kavin  91.0
```

## 6. Function pointer used to call two functions

**Question:** Define two functions with the same signature. Declare a function pointer and use the pointer to call both functions.

### *Program*

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}

int main() {
    int (*fp)(int, int);

    fp = add;
    printf("Addition = %d\\n", fp(6, 4));

    fp = multiply;
    printf("Multiplication = %d\\n", fp(6, 4));

    return 0;
}
```

### *Output*

```
Addition = 10
Multiplication = 24
```

## 7. Integer pointer and double pointer

**Question:** Declare an integer, an integer pointer, and a double pointer. Print the values and addresses at each level.

### *Program*

```
#include <stdio.h>

int main() {
    int x = 25;
    int *p = &x;
    int **q = &p;

    printf("Value of x       = %d\\n", x);
    printf("Address of x      = %p\\n", (void *)&x);

    printf("Value using p     = %d\\n", *p);
    printf("Address in p      = %p\\n", (void *)p);
    printf("Address of p      = %p\\n", (void *)&p);

    printf("Value using q     = %d\\n", **q);
    printf("Value of p via q  = %p\\n", (void *)*q);
    printf("Address of q      = %p\\n", (void *)&q);

    return 0;
}
```

### *Output*

```
Value of x       = 25
Address of x      = 0x7ffd...
Value using p     = 25
Address in p      = 0x7ffd...
Address of p      = 0x7ffd...
Value using q     = 25
Value of p via q  = 0x7ffd...
Address of q      = 0x7ffd...

Note: Memory addresses will be different on each computer/run.
```

## 8. Structure Student and sorting based on a field

**Question:** Define a structure Student with id, name and grade. Create an array of student structures and sort them based on a field (grade).

### *Program*

```
#include <stdio.h>
#include <string.h>

struct Student {
    int id;
    char name[20];
    float grade;
};

int main() {
    struct Student s[] = {
        {101, "Arun", 78.5},
        {102, "Bala", 92.0},
        {103, "Kavin", 85.5},
        {104, "Dinesh", 70.0}
    };

    int n = sizeof(s) / sizeof(s[0]);
    struct Student temp;

    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (s[i].grade > s[j].grade) {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    printf("Students sorted by grade:\\n");
    for (int i = 0; i < n; i++)
        printf("%d  %s  %.1f\\n",
               s[i].id, s[i].name, s[i].grade);

    return 0;
}
```

### *Output*

```
Students sorted by grade:
104  Dinesh  70.0
101  Arun    78.5
103  Kavin   85.5
102  Bala    92.0
```

## 9. Sort negative numbers using radix sort

**Question:** Sort a set of negative numbers using radix sort.

### *Program*

```
#include <stdio.h>

int getMaxAbs(int a[], int n) {
    int max = -a[0];
    for (int i = 1; i < n; i++)
        if (-a[i] > max)
            max = -a[i];
    return max;
}

void countSortAbs(int a[], int n, int exp) {
    int output[20];
    int count[10] = {0};

    for (int i = 0; i < n; i++)
        count[(-a[i] / exp) % 10]++;

    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];

    for (int i = n - 1; i >= 0; i--) {
        int digit = (-a[i] / exp) % 10;
        output[count[digit] - 1] = a[i];
        count[digit]--;
    }

    for (int i = 0; i < n; i++)
        a[i] = output[i];
}

void radixSortNegative(int a[], int n) {
    int max = getMaxAbs(a, n);

    for (int exp = 1; max / exp > 0; exp *= 10)
        countSortAbs(a, n, exp);

    /* Reverse because larger absolute value means smaller negative number. */
    for (int i = 0, j = n - 1; i < j; i++, j--) {
        int temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }
}

int main() {
    int a[] = {-45, -12, -89, -34, -67};
    int n = sizeof(a) / sizeof(a[0]);

    radixSortNegative(a, n);

    printf("Sorted negative numbers: ");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\\n");
    return 0;
}
```

### *Output*

```
Sorted negative numbers: -89 -67 -45 -34 -12
```

## 10. Copy one array to another using pointers

**Question:** Write a C program to copy one array to another using pointers.

### *Program*

```
#include <stdio.h>

int main() {
    int source[] = {10, 20, 30, 40, 50};
    int destination[5];

    int *p = source;
    int *q = destination;
    int n = sizeof(source) / sizeof(source[0]);

    for (int i = 0; i < n; i++)
        *(q + i) = *(p + i);

    printf("Source array      : ");
    for (int i = 0; i < n; i++)
        printf("%d ", *(p + i));

    printf("\\nCopied array      : ");
    for (int i = 0; i < n; i++)
        printf("%d ", *(q + i));

    printf("\\n");
    return 0;
}
```

### *Output*

```
Source array      : 10 20 30 40 50
Copied array      : 10 20 30 40 50
```